SQL WINDOW FUNCTIONS

Guide Pratique Complet

Basé sur le schéma CO – E-Commerce Database

Tables du projet

CO.CUSTOMERS · CO.ORDERS · CO.ORDER_ITEMS · CO.PRODUCTS · CO.INVENTORY · CO.SHIPMENTS · CO.STORES

---

0. Pourquoi les Window Functions ?

Les fonctions de fenêtrage permettent de faire des calculs sur un ensemble de lignes en relation avec la ligne courante — sans réduire le nombre de lignes comme le fait GROUP BY.

Comparaison rapide :

Approche	Résultat	Lignes retournées
GROUP BY + SUM()	Total par groupe	1 ligne par groupe
SUM() OVER (PARTITION BY)	Total par groupe + détail	Toutes les lignes

GROUP BY vs Window Function

La syntaxe de base est toujours :

```
fonction() OVER (
    PARTITION BY colonne_de_groupement   -- optionnel
    ORDER BY    colonne_de_tri           -- optionnel
    ROWS/RANGE BETWEEN ...               -- optionnel
)
```

---

1. ROW_NUMBER, RANK, DENSE_RANK

1.1 ROW_NUMBER() — Numérotation unique

Cas pratique : Numéroter les commandes de chaque client par date.

```
SELECT
    o.customer_id,
    o.order_id,
```

```
    o.order_date,
    o.order_total,
    ROW_NUMBER() OVER (
        PARTITION BY o.customer_id
        ORDER BY     o.order_date ASC
    ) AS num_commande
FROM co.orders o
ORDER BY o.customer_id, o.order_date;
```

customer_id	order_id	order_date	order_total	num_commande
C001	O-1001	2024-01-05	120.00	1
C001	O-1045	2024-02-18	340.00	2
C001	O-1098	2024-03-30	89.00	3
C002	O-1003	2024-01-07	210.00	1
C002	O-1072	2024-03-11	450.00	2

ROW_NUMBER par client – chaque client repart de 1

ROW_NUMBER() est toujours unique, même en cas d'ex-aequo.

1.2 RANK() vs DENSE_RANK() — Classements avec ex-aequo

Cas pratique : Classer les produits par chiffre d'affaires total.

```
SELECT
    p.product_name,
    SUM(oi.unit_price * oi.quantity)       AS ca_total,
    RANK()       OVER (ORDER BY SUM(oi.unit_price * oi.quantity) DESC) AS
rank_std,
    DENSE_RANK() OVER (ORDER BY SUM(oi.unit_price * oi.quantity) DESC) AS
rank_dense
FROM co.order_items oi
JOIN co.products    p  ON p.product_id = oi.product_id
GROUP BY p.product_name
ORDER BY ca_total DESC;
```

product_name	ca_total	rank_std	rank_dense
Laptop Pro X	85,200	1	1
Laptop Pro X2	85,200	1	1
Wireless Mouse	42,000	3	2
USB Hub	38,500	4	3

RANK saute de 1→3, DENSE_RANK va de 1→2 (pas de saut)

1.3 Application : Top-N par groupe

Cas pratique : Les 2 meilleurs produits par catégorie (pattern classique avec CTE).

```
WITH ranked_products AS (
    SELECT
        p.category,
        p.product_name,
        SUM(oi.unit_price * oi.quantity)  AS ca,
        RANK() OVER (
            PARTITION BY p.category
```

```
            ORDER BY SUM(oi.unit_price * oi.quantity) DESC
        ) AS rang
    FROM co.order_items oi
    JOIN co.products    p ON p.product_id = oi.product_id
    GROUP BY p.category, p.product_name
)
SELECT category, product_name, ca, rang
FROM   ranked_products
WHERE  rang <= 2
ORDER BY category, rang;
```

💡 C'est l'un des patterns les plus utilisés en reporting. Retenez-le !

---

2. LAG() et LEAD() — Comparaison temporelle

2.1 LAG() — Valeur de la ligne précédente

Cas pratique : Calculer l'évolution du montant entre chaque commande d'un client.

```
SELECT
    customer_id,
    order_id,
    order_date,
    order_total,
    LAG(order_total) OVER (
        PARTITION BY customer_id
        ORDER BY     order_date
    )                                          AS commande_precedente,
    order_total - LAG(order_total) OVER (
        PARTITION BY customer_id
        ORDER BY     order_date
    )                                          AS evolution
FROM co.orders
ORDER BY customer_id, order_date;
```

customer_id	order_date	order_total	commande_precedente	evolution
C001	2024-01-05	120.00	NULL	NULL
C001	2024-02-18	340.00	120.00	+220.00
C001	2024-03-30	89.00	340.00	-251.00
C002	2024-01-07	210.00	NULL	NULL
C002	2024-03-11	450.00	210.00	+240.00

LAG() retourne NULL pour la 1ère ligne de chaque partition

2.2 LEAD() — Valeur de la ligne suivante

Cas pratique : Calculer le délai entre deux expéditions consécutives d'une commande.

```
SELECT
    order_id,
    shipment_date,
    status,
```

```
    LEAD(shipment_date) OVER (
        PARTITION BY order_id
        ORDER BY     shipment_date
    )                                          AS prochaine_expedition,
    DATEDIFF(
        LEAD(shipment_date) OVER (
            PARTITION BY order_id ORDER BY shipment_date
        ),
        shipment_date
    )                                          AS jours_entre_expeditions
FROM co.shipments
ORDER BY order_id, shipment_date;
```

💡 LAG(col, N, default) et LEAD(col, N, default) acceptent un offset N et une valeur par défaut au lieu de NULL.

---

3. Agrégats glissants : SUM, AVG, COUNT, MIN, MAX

3.1 Cumul progressif (Running Total)

Cas pratique : Calculer le chiffre d'affaires cumulé au fil du temps.

```
SELECT
    DATE_TRUNC('month', order_date)            AS mois,
    SUM(order_total)                           AS ca_mensuel,
    SUM(SUM(order_total)) OVER (
        ORDER BY DATE_TRUNC('month', order_date)
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    )                                          AS ca_cumule
FROM co.orders
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY mois;
```

mois	ca_mensuel	ca_cumule
2024-01	45,200	45,200
2024-02	68,400	113,600
2024-03	52,100	165,700
2024-04	71,300	237,000

Le ca_cumule s'additionne de mois en mois

3.2 Moyenne mobile (Rolling Average)

Cas pratique : Moyenne des ventes sur les 3 derniers mois glissants.

```
SELECT
    DATE_TRUNC('month', order_date)            AS mois,
    SUM(order_total)                           AS ca_mensuel,
    AVG(SUM(order_total)) OVER (
        ORDER BY DATE_TRUNC('month', order_date)
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    )                                          AS moy_mobile_3mois
FROM co.orders
```

```
GROUP BY DATE_TRUNC('month', order_date)
ORDER BY mois;
```

💡 ROWS BETWEEN 2 PRECEDING AND CURRENT ROW = les 2 lignes avant + la ligne actuelle = fenêtre de 3 lignes.

3.3 Part du total (Pourcentage)

Cas pratique : Part de chaque magasin dans le CA global.

```
SELECT
    s.store_name,
    SUM(o.order_total)                              AS ca_store,
    SUM(SUM(o.order_total)) OVER ()                 AS ca_global,
    ROUND(
        100.0 * SUM(o.order_total)
              / SUM(SUM(o.order_total)) OVER (),
        2
    )                                               AS pct_ca
FROM co.orders   o
JOIN co.stores   s ON s.store_id = o.store_id
GROUP BY s.store_name
ORDER BY ca_store DESC;
```

store_name	ca_store	ca_global	pct_ca
Paris Centre	98,400	237,000	41.52%
Lyon Sud	72,100	237,000	30.42%
Bordeaux	66,500	237,000	28.06%

OVER () sans PARTITION = fenêtre = toute la table

---

4. FIRST_VALUE, LAST_VALUE, NTH_VALUE

4.1 FIRST_VALUE() — Première valeur de la fenêtre

Cas pratique : Afficher le prix du premier produit commandé par client.

```
SELECT
    o.customer_id,
    o.order_id,
    o.order_date,
    o.order_total,
    FIRST_VALUE(o.order_total) OVER (
        PARTITION BY o.customer_id
        ORDER BY     o.order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    )  AS premiere_commande_montant
FROM co.orders o
ORDER BY o.customer_id, o.order_date;
```

💡 Toujours préciser ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING avec LAST_VALUE, sinon la fenêtre par défaut s'arrête à la ligne courante !

4.2 NTH_VALUE() — La N-ième valeur

Cas pratique : Toujours afficher le montant de la 2e commande de chaque client sur toutes les lignes.

```
SELECT
    customer_id,
    order_id,
    order_date,
    order_total,
    NTH_VALUE(order_total, 2) OVER (
        PARTITION BY customer_id
        ORDER BY     order_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    )  AS montant_2eme_commande
FROM co.orders
ORDER BY customer_id, order_date;
```

---

5. Distribution : NTILE, PERCENT_RANK, CUME_DIST

5.1 NTILE(N) — Diviser en N groupes égaux

Cas pratique : Segmenter les clients en 4 quartiles selon leur CA total (segmentation RFM simplifiée).

```
WITH ca_clients AS (
    SELECT
        customer_id,
        SUM(order_total) AS ca_total
    FROM co.orders
    GROUP BY customer_id
)
SELECT
    customer_id,
    ca_total,
    NTILE(4) OVER (ORDER BY ca_total DESC) AS quartile,
    CASE NTILE(4) OVER (ORDER BY ca_total DESC)
        WHEN 1 THEN 'Top clients (Q1)'
        WHEN 2 THEN 'Bons clients (Q2)'
        WHEN 3 THEN 'Clients moyens (Q3)'
        WHEN 4 THEN 'Petits clients (Q4)'
    END AS segment
FROM ca_clients
ORDER BY ca_total DESC;
```

5.2 PERCENT_RANK() et CUME_DIST()

Cas pratique : Calculer le percentile de chaque commande dans la distribution globale.

```
SELECT
    order_id,
    order_total,
```

```
    ROUND(PERCENT_RANK() OVER (ORDER BY order_total) * 100, 1) AS
percentile_rank,
    ROUND(CUME_DIST()   OVER (ORDER BY order_total) * 100, 1) AS
cume_dist_pct
FROM co.orders
ORDER BY order_total;
```

order_id	order_total	percentile_rank	cume_dist_pct
O-1001	89.00	0.0%	5.0%
O-1045	120.00	5.0%	15.0%
O-1003	210.00	45.0%	55.0%
O-1072	340.00	75.0%	85.0%
O-1098	450.00	100.0%	100.0%

PERCENT_RANK commence à 0, CUME_DIST commence à >0

---

6. Cas avancé : Pipeline analytique complet

Objectif : Construire un tableau de bord par client avec toutes les métriques en une seule requête.

```
WITH client_orders AS (
    SELECT
        o.customer_id,
        o.order_id,
        o.order_date,
        o.order_total,
        -- Numéro de commande
        ROW_NUMBER() OVER (PARTITION BY o.customer_id ORDER BY o.order_date)
AS n_cmd,
        -- Montant commande précédente
        LAG(o.order_total)  OVER (PARTITION BY o.customer_id ORDER BY
o.order_date) AS prev_total,
        -- CA cumulé par client
        SUM(o.order_total)  OVER (PARTITION BY o.customer_id ORDER BY
o.order_date
                                  ROWS BETWEEN UNBOUNDED PRECEDING AND
CURRENT ROW) AS ca_cumule,
        -- Rang dans tous les clients
        RANK() OVER (ORDER BY SUM(o.order_total) OVER (PARTITION BY
o.customer_id)) AS rang_global
    FROM co.orders o
),
enriched AS (
    SELECT *,
        ROUND(100.0 * (order_total - prev_total) / NULLIF(prev_total,0), 1)
AS evol_pct,
        NTILE(3) OVER (ORDER BY ca_cumule DESC) AS segment
    FROM client_orders
)
SELECT
    customer_id,
```

```
    order_id,
    order_date,
    order_total,
    n_cmd,
    evol_pct,
    ca_cumule,
    rang_global,
    CASE segment WHEN 1 THEN 'Premium' WHEN 2 THEN 'Standard' ELSE 'Basic'
END AS segment_label
FROM enriched
ORDER BY customer_id, order_date;
```

💡 Imbriquer plusieurs window functions dans une CTE est un pattern très puissant pour les dashboards analytiques.

---

7. Cheat Sheet — Référence rapide

Fonction	Utilité	Exemple typique
ROW_NUMBER()	Numérotation unique	Pagination, déduplication
RANK()	Classement avec sauts	Top-N avec ex-aequo
DENSE_RANK()	Classement sans saut	Médailles (or/argent/bronze)
LAG(col, N)	Valeur N lignes avant	Évolution vs période précédente
LEAD(col, N)	Valeur N lignes après	Délai avant prochaine étape
SUM() OVER()	Cumul ou total global	Running total, % du total
AVG() OVER()	Moyenne glissante	Tendance lissée
FIRST_VALUE()	1ère valeur de fenêtre	Première commande client
LAST_VALUE()	Dernière valeur	Dernier statut connu
NTH_VALUE(col, N)	N-ième valeur	Benchmarking
NTILE(N)	Découpage en N groupes	Quartiles, déciles
PERCENT_RANK()	Percentile (0 à 1)	Positionnement relatif
CUME_DIST()	Distribution cumulée	Proportion en dessous

Récapitulatif de toutes les fonctions de fenêtrage

Bonne pratique des Window Functions ! 🚀

github.com/ton-repo/sql-window-functions